

Appendix S7. Computational environment and reproduction workflow

ENA-ABM v0.5 main theoretical experiment

This appendix provides an executable route from the archived source package to two independently generated result directories and a formal reproduction check. It complements Appendix S6, which reports the completed verification evidence. The procedure reproduces a theoretical computational experiment with diagnostic parameters; it does not create empirical calibration or external validation.

S7.1 Required materials and execution boundary

The workflow begins from the root directory of ENA_ABM_v0.5_reproducible.zip. Extract the S7 companion package into that root so its requirements file, wrapper, and two helper scripts merge into the documented directory structure. Commands must be executed from that directory because the scripts resolve configs/, docs/, src/, tests/, and outputs/ relative to the package root. Use new or empty output directories for independent reproductions. The archived package retains the complete run 1 results and the run 2 manifest and hash registry; a new audit should regenerate both runs rather than treat the archived outputs as inputs.

The primary configuration is configs/main_study_v0.5.json. Its recorded experiment seed is 2026072505; it specifies 96 profiles, 400 paired replications per cell, 500 bootstrap samples, and a refined relative-cost grid. Do not edit the configuration, frozen source, preregistered design, or seeds when attempting an exact reproduction.

S7.2 Software requirements

Table S7.1. Software and execution requirements

Component	Requirement	Reproduction role
Python	3.11 or later	Declared by pyproject.toml; the S7 preflight was verified under Python 3.12.13.
Core package	Local ena_abm source under src/	Implements the frozen model and command-line interface.
NumPy	2.3.5 in the S7 reference environment	Numerical arrays, quantiles, and deterministic profile construction.
SciPy	1.17.0 in the S7 reference environment	Spearman rank analysis.
pandas	2.2.3 in the S7 reference environment	Sensitivity-analysis data preparation.
scikit-learn	1.8.0 in the S7 reference environment	Cross-validation, random forest, permutation importance, and descriptive tree.
Matplotlib	3.10.8 in the S7 reference environment	Generation of the four principal figures.
Parallel execution	Four workers by default	Reduces runtime; deterministic seed namespaces make scientific results independent of task completion order.
Writable graphics cache	MPLCONFIGDIR	Prevents cache and multiprocessing problems in restricted environments.

The original v0.5 archive declares Python ≥ 3.11 but leaves dependencies=[] and does not retain a lock file. The versions above therefore document the environment in which this appendix's preflight was verified on 24 August 2026; they are not claimed to be the unrecorded original simulation environment. Before public release, the repository should retain this requirements snapshot and preferably an additional full environment export. Scientific summaries should be checked even when operating-system, compression-library, font, or graphics differences prevent cross-platform byte identity.

S7.3 Environment setup

The following POSIX-shell commands create an isolated environment. PowerShell users should activate `.venv\Scripts\Activate.ps1` and set environment variables with `$env:NAME='value'`.

```
python -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
python -m pip install -r requirements_reproduction_v0.5.txt
python -m pip install -e .
export PYTHONPATH=src
export MPLCONFIGDIR=/tmp/ena_abm_mpl
mkdir -p "$MPLCONFIGDIR"
```

The companion requirements file is deliberately kept outside the frozen source inventory. Adding it to the public repository improves execution documentation but must not be represented as part of the preregistered model source.

S7.4 Preflight verification

Run the automated suite and frozen-source comparison before starting the expensive experiment:

```
PYTHONPATH=src python -m unittest discover -s tests -v
python scripts/verify_frozen_v0_4.py
```

Expected results are 64 tests executed with zero failures and zero errors, followed by PASS: 15 frozen v0.4 model-source files match. These commands were rerun successfully while preparing S7. A failure at this stage invalidates an exact reproduction attempt and should be resolved before any simulation is interpreted.

S7.5 Two complete executions

Run the first execution in a clean directory:

```
MPLCONFIGDIR=/tmp/ena_abm_mpl PYTHONPATH=src \
python scripts/run_main_study_v0_5.py \
  --config configs/main_study_v0.5.json \
  --output outputs/main_study_v0_5/run1 \
  --workers 4
```

Repeat the same command with `--output outputs/main_study_v0_5/run2`. The `--workers` value may be adjusted to available hardware. Do not use `--resume-main` for an independent reproduction because that option intentionally reuses an existing main grid, threshold table, and bootstrap.

Each successful execution should finish with a JSON status record containing `"status": "completed"`, `"total_paired_rows": 972800`, and `"interpretation_blocked": false`. The run manifest should additionally record `"principal_files": 34` and `"parameters_empirically_calibrated": false`.

S7.6 Canonicalization and manifest rebuilding

The current v0.5 raw-data writer already creates gzip headers with `mtime=0`. The following idempotent step is retained to normalize files produced by any earlier timestamp-bearing writer:

```
python scripts/canonicalize_gzip_v0_5.py \
  outputs/main_study_v0_5/run1 outputs/main_study_v0_5/run2
```

Because canonicalization changes compressed container bytes, per-run hash registries must be rebuilt afterward. The companion script performs that bookkeeping without rerunning the simulations:

```
python scripts/rebuild_run_manifest_v0_5.py \
  outputs/main_study_v0_5/run1 configs/main_study_v0.5.json
python scripts/rebuild_run_manifest_v0_5.py \
  outputs/main_study_v0_5/run2 configs/main_study_v0.5.json
```

This operation does not change decompressed CSV content, statistical results, or interpretation. It only aligns the recorded byte hashes with the canonical compressed files.

S7.7 Reproduction comparison and acceptance checks

```
python scripts/compare_main_reproductions_v0_5.py \
  outputs/main_study_v0_5/run1 \
  outputs/main_study_v0_5/run2
```

```
python scripts/check_expected_outputs_v0_5.py \
  outputs/main_study_v0_5/run1 \
  outputs/main_study_v0_5/run2 \
  outputs/main_study_v0_5/reproduction_comparison_v0.5.csv
```

Table S7.2. Minimum acceptance criteria

Check	Expected value	Evidence
Automated tests	64/64 passed	Test runner output
Frozen source	15/15 SHA-256 matches	verify_frozen_v0_4.py output
Paired rows per execution	972,800	Each manifest_v0.5.json
Principal outputs per execution	34	Each manifest and principal_hashes_v0.5.csv
Logical identity	34/34	identical=1 in the comparison CSV
Byte identity after canonicalization	34/34	binary_identical=1 in the comparison CSV
Interpretation stop gate	Not activated	interpretation_blocked=false
Calibration status	Diagnostic, not empirical	parameters_empirically_calibrated=false

The comparison is accepted only when all 34 records satisfy both logical and binary identity after canonicalization. If only logical identity holds, the researcher should first inspect gzip headers, software versions, and figure-rendering differences rather than infer a scientific discrepancy.

S7.8 Output map and preservation

Each run produces 34 principal artifacts: 16 uncompressed result tables, seven compressed raw-data tables, four JSON manifests or diagnostics, three Markdown reports or rule files, and four figures. Together they preserve thresholds, summaries, sensitivity, robustness, paired simulation records, study metadata, and article-ready visualizations.

Preserve the configuration, preregistered design, source hashes, both run manifests, both principal-hash registries, and the reproduction-comparison CSV alongside any public release. The GitHub repository should also record the operating system, Python and package versions, hardware or worker count, execution date, and commit identifier. These additions strengthen future cross-machine reproducibility without changing the frozen v0.5 design.

The companion package contains the requirements snapshot, POSIX wrapper, manifest rebuilder, acceptance checker, original README and pyproject.toml, and main configuration. The commands shown here remain authoritative.